

Lightning lesson, sixty minutes

Enforce Code Quality with Automated Agent Verifiers

The session that changes how people review. Most of the hour is spent on one idea: undo the change and watch the test go red.

Fifteen minutes ahead

Arm the demonstration

Goal A recorded run is ready that produced a genuinely flawed change, so the rejection is real rather than staged.

Say Nothing. Loading a recording.

Run

```
$ make demo STEP=04
```

Check

- The gate prints its result line and nothing summarises it afterwards.
- The rejection you intend to show is in the recording, with its reason.

Ask *Nobody. Have the flawed diff open in a second window, ready to show.*

0:00 to 0:10

Code that reads correctly

Goal	The room agrees that reviewing plausible work is more expensive than reviewing obviously broken work.
Say	• Put the flawed diff on screen with no commentary and give them thirty seconds to find the problem. • Most will not, and that is the lesson. Reveal it, then point out that the tests were green. • Draw the conclusion carefully: the problem is not that the work was bad, it is that nothing independent had looked before you did.
Run	The diff on screen. No commands.
Room sees	A change that looks fine, with a passing suite beside it.
Ask	<i>How much of your review time goes on work that turns out to be correct?</i>

0:10 to 0:24

One line nobody may rephrase

Goal Attendees can state why a missing check must be louder than a failing one.

Say

- Show the script ending in a single result line and explain the rule around it: quoted exactly, never summarised.
- Delete the test command in front of them and run it again. The result is not green and it is not a failure either.
- Make the argument explicit. Absence and success look identical from the outside, and only one of them is safe to act on.

Run

```
$ bash steps/04-verification/gates.sh
$ bash steps/04-verification/gates.sh --without-tests
```

The script is `steps/04-verification/gates.sh`.

Room sees

- A passing result line, then a misconfigured one with a different exit code.
- No prose anywhere claiming the run was fine.

Ask *If your pipeline lost its test step tonight, how long before anyone noticed?*

0:24 to 0:42

Undo the fix

Goal Everyone leaves able to apply this to code that has no agents anywhere near it.

- Say**
- State the check in one sentence and then do it, rather than explaining it twice.
 - Show the good case, then plant a test that passes whatever the code does and show it surviving. That contrast is the teaching moment.
 - Say the general version out loud, because it is the most portable idea in the course: a test that has never failed is not known to test anything.
-

Run

```
$ node steps/04-verification/demo.ts negative-proof
$ node steps/04-verification/demo.ts negative-proof --toothless
```

- Room sees**
- The change reverted, the new test going red, the change restored.
 - The second run reporting that the test stayed green while its subject was broken.
-

Ask *Take the last test you wrote. Would it have gone red without your change?*

0:42 to 0:54

Reading the diff cold

Goal They understand why the reviewer is deliberately denied the story of the work.

- Say**
- Describe the anchoring problem from human review, which everyone recognises, then apply it to a second agent.
 - Show what the reviewer receives and, more importantly, what it does not receive.
 - Return to the flawed diff from the opening and let the cold reviewer find it. That closes the hour on evidence rather than assertion.
-

Run

```
$ node steps/04-verification/demo.ts verify
```

- Room sees**
- The reviewer's input: the task and the diff, with no account of the attempt.
 - A rejection naming the behaviour the change added that the task never asked for.
-

Ask *What does your reviewer currently read first, and does it change what they conclude?*

0:54 to 1:00

Cost and close

Goal Nobody leaves thinking this is free.

Say

- Be honest about the price. An independent pass adds minutes and sometimes rejects work that was fine.
- Give the mitigation rather than the excuse: bound the stage, and let it consume evidence the implementation already produced instead of repeating it.

Run Show the stage timings from the recorded run.

Room sees Where the minutes actually went, with checking larger than writing.

Ask *What would you accept paying, in minutes, for an independent read of every change?*

0:35 to 0:45

A reviewer that can say no

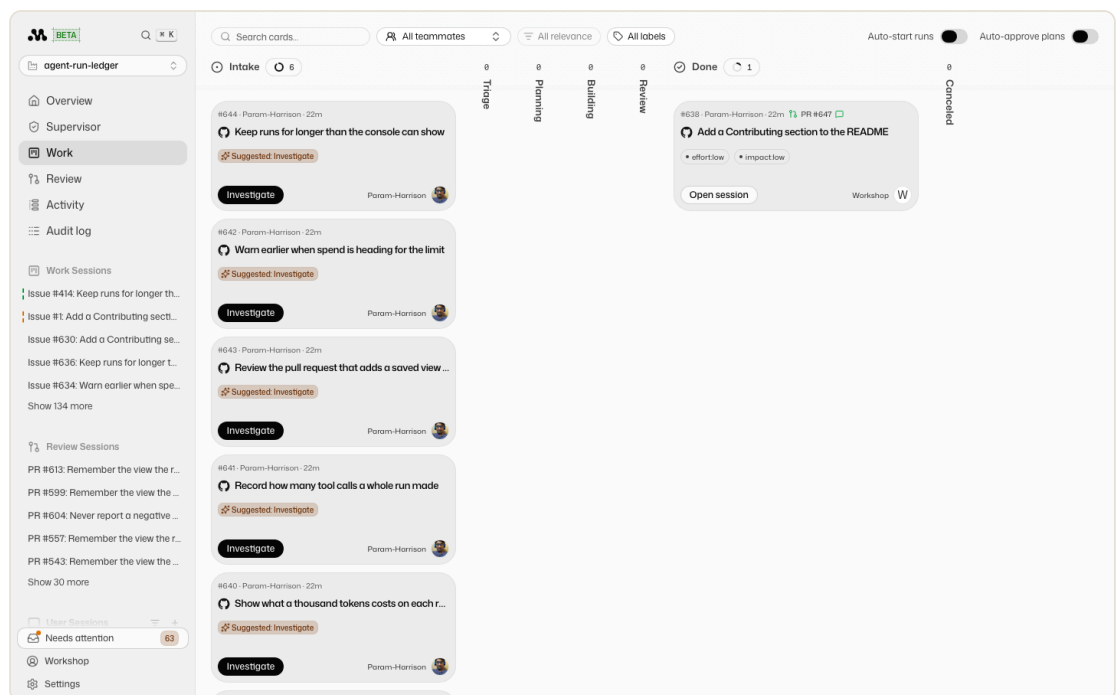
Goal They accept that a reviewer which never rejects anything has not been tested.

Say

- The interesting artefact is the refusal, not the approval.
- Read the two findings aloud and note that one of them is a weakened assertion, which no path rule could have caught.
- This is why the charter forbids editing a test to make a suite green: it is a property of the change, not of a filename.

Run

```
$ make lab-reset
```



The verdict line, with the two findings that produced it.

Show on the board

Review to the pull request to Conversation to the verdict line

The verdict is the first line, on purpose

A reviewer that buries its conclusion in prose cannot be checked by anything. One line, one of two values, and everything after it is the argument for it.

A weakened assertion is not a path rule

Changing `toBeCloseTo(1, 10)` to `toBeGreaterThanOrEqual(0)` touches no protected file. No glob catches it, which is why the charter forbids it in words and the reviewer has to be the thing that notices.

A picture is the one artefact a test cannot read

Four screenshots in this repository were a loading spinner, and every assertion around them passed. Show the room the guard that refuses to photograph a page still saying it is loading.

Room does

- Reads the verdict line and both findings.
- Identifies which finding a glob could have prevented and which it could not.

Ask

Your suite went green after an edit to a test. What would tell you which kind of edit it was?

Concepts: the verdict line, holdout tests, the cold reviewer, the negative proof, Doer and Tester.